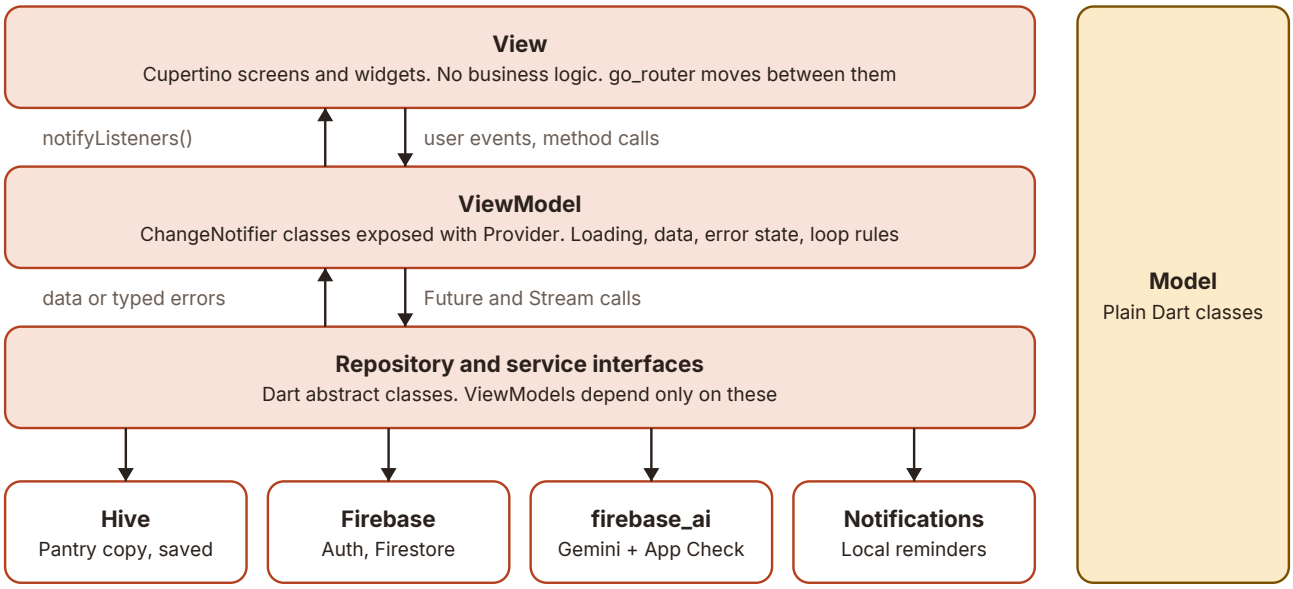
Architecture and Protocols

Dolaptan · MTM4681 Mobile Programming · Milestone · Team [TAKIM NO] · [AD SOYAD 1], [AD SOYAD 2]

1. Software design pattern: MVVM with Provider

Dolaptan uses **Model-View-ViewModel**, the clean structure the course asks for. Each ViewModel is a `ChangeNotifier` exposed with **Provider**, and widgets rebuild when it notifies. We chose Provider over Riverpod because it is simpler and enough for a two-person team on a short schedule.



Layer	Role in Dolaptan	Examples
Model	Plain Dart classes with JSON and Hive conversion	PantryItem, ScannedItem, Recipe, ListItem, Preferences, Household
View	Display state and forward events. They never call Firebase	PantryScreen, ReviewItemsScreen, RecipeDetailScreen
ViewModel	Screen state and the loop rules: Cooked lowers items, Bought restocks them	PantryViewModel, ScanViewModel, RecipeViewModel, ShoppingListViewModel
Repository / Service	Hide where data comes from, so ViewModels can be tested with fakes	PantryRepository, RecipeAiService, ReminderService

2. Screen map

Screen	go_router path	ViewModel	Main data
Login	/login	AuthViewModel	Firebase Auth
Pantry (tab)	/pantry	PantryViewModel	Firestore stream, Hive copy, offline banner
Edit item (sheet)	/pantry/:id	PantryViewModel	State, expiry date, reminder
Scan	/scan	ScanViewModel	Camera image, AI detection
New items	/scan/review	ScanViewModel	Detected items, expiry dates
Preferences (sheet)	/cook	RecipeViewModel	Time, servings, allergies
Recipe suggestions	/cook/results	RecipeViewModel	3 AI recipes, expiring first
Recipe detail	/recipe/:id	RecipeViewModel	Save, add to list, Cooked
Recipes (tab)	/recipes	SavedViewModel	Saved recipes in Hive
Shopping list (tab)	/list	ShoppingListViewModel	Firestore stream, Bought restocks

A `redirect` in go_router sends signed-out users to /login. A `StatefulShellRoute` keeps the three tabs. Offline status is a banner inside the Pantry screen. Scan uses an `AnimationController` for the moving line. Screens open with iOS push transitions (`CupertinoPage`), sheets slide up, and the recipe title moves into the detail screen with a `Hero`.

UI layer: Cupertino widgets on iOS and Android

The interface follows iOS design, so the View layer uses Flutter's Cupertino widgets. They now ship in the **cupertino_ui** package (version 1.1.5 on pub.dev, published by flutter.dev), which supports both Android and iOS. The same widgets render on both platforms, so one codebase gives the same look.

Design element	Flutter widget
App shell and routing	CupertinoApp.router with go_router, CupertinoPage for push transitions
Large-title navigation bar	CupertinoSliverNavigationBar
Tab bar	CupertinoTabBar inside a StatefulShellRoute
Inset grouped list	CupertinoListSection.insetGrouped, CupertinoListTile
Edit item, Preferences	showCupertinoSheet
Item state, time, servings	CupertinoSlidingSegmentedControl
Expiry date, reminders	CupertinoDatePicker, CupertinoSwitch
Cooked confirmation	CupertinoAlertDialog
Search, loading	CupertinoSearchTextField, CupertinoActivityIndicator
Swipe to delete	flutter_slidable (MIT, Android and iOS)
Icons	cupertino_icons (MIT)

Android: SafeArea keeps content clear of the Android status and navigation bars, and the system Back gesture pops the go_router stack. Inter is bundled as the app font, because Apple's SF Pro license does not allow use on non-Apple systems. The cupertino_ui page does not mention Liquid Glass, so bars use a plain translucent background. **[TO CHECK: Liquid Glass support in cupertino_ui before week 11.]**

3. Data model

```
Dart models
  PantryItem   id, name, state (have | low | out), expiresOn?, remind, addedBy, updatedAt
  ScannedItem  name, isUncertain, expiresOn?
  Preferences  minutes, servings, allergies[]
  Recipe       id, title, minutes, servings, usedIngredients[], usesExpiring[],
               missingIngredients[], steps[], photoPath? (photo space hidden when empty)
  ListItem     id, name, addedBy, addedByName, checked, createdAt

Firestore (shared by the household)
  users/{uid}                        name, householdId
  households/{id}                    name, inviteCode, memberIds[]
  households/{id}/pantry/{itemId}    name, state, expiresOn, addedBy, updatedAt
  households/{id}/items/{itemId}     name, addedBy, addedByName, checked, createdAt
  inviteCodes/{code}                 householdId

Hive (device)
  pantry_{householdId}               last synced pantry, opens offline
  saved_{uid}                        saved recipes
  prefs                              last used preferences
```

Loop rules. Cooked: every used item goes from have to low, or from low to out. Items that run out leave the pantry and join the shopping list in one batched write. Bought: the item returns to the pantry as have and leaves the list in one batched write. When two people edit the same item, the last write wins, which is enough for a household.

Photos are compressed, sent only for analysis, and never stored. Firestore security rules allow only household members to read or change that household's pantry and list. A user joins by reading one invite code document and can add only their own uid to memberIds.

4. Key interfaces (Dart abstract classes)

Dart has no `protocol` keyword, so abstract classes define these contracts. ViewModels depend on them rather than on Firebase, so an implementation can be replaced by a fake in tests, or the AI provider can change without touching the UI.

```
abstract class AuthService {
  Stream<AppUser?> get userChanges;
  Future<void> signIn(String email, String password);
  Future<void> signUp(String email, String password, String name);
  Future<void> signOut();
}
```

```
abstract class PantryRepository {
  Stream<List<PantryItem>> watch(String householdId);
  Future<void> addAll(String householdId, List<ScannedItem> items);
  Future<void> update(String householdId, PantryItem item);
  Future<void> markCooked(String householdId, Recipe recipe);  // one batched write
  Future<void> remove(String householdId, String itemId);
}

abstract class ShoppingListRepository {
  Stream<List<ListItem>> watchItems(String householdId);
  Future<void> add(String householdId, String name);
  Future<void> markBought(String householdId, ListItem item); // also puts it in the pantry
}

abstract class RecipeAiService {
  Future<List<ScannedItem>> detectItems(Uint8List photo);
  Future<List<Recipe>> suggestRecipes(List<PantryItem> pantry, Preferences prefs);
}

abstract class SavedRecipeRepository {
  Stream<List<Recipe>> watchAll();
  Future<void> save(Recipe recipe);
  Future<void> delete(String recipeId);
}

abstract class ReminderService {
  Future<void> schedule(PantryItem item);   // the day before expiresOn
  Future<void> cancel(String itemId);
}

abstract class HouseholdRepository {
  Future<Household> create();
  Future<Household> joinWithCode(String inviteCode);
}
```

5. Communication protocols

Protocol / technology	Purpose in the app
HTTPS (TLS)	Encrypts traffic between the app and Firebase services
Firebase Authentication	Signs users in with email and password and gives requests a verified identity
Firestore real-time listeners	Push pantry and list changes to every household member as Dart Streams
Firestore batched writes	Keep the loop consistent: Cooked and Bought change pantry and list together
Firestore security rules	Check on the server who can read or change each document
Firebase AI Logic (firebase_ai)	Sends the photo or pantry list to the Gemini Developer API through Firebase, so no Gemini API key ships in the app. Planned model: gemini-3.8-flash, kept in one constant
Firebase App Check	Proves that AI requests come from our real app. Play Integrity on Android, debug provider in development
JSON with a response schema	Requests structured output (responseMimeType + responseSchema), then parses it into Dart models
Local notifications	flutter_local_notifications schedules expiry reminders on the device. Android 13 and later asks for permission

6. Main sequences

Add food: (1) The photo goes to ScanViewModel. (2) RecipeAiService returns items, and unclear ones are marked uncertain. (3) The user fixes the list and adds expiry dates. (4) PantryRepository saves them for the household, and ReminderService schedules reminders.

Cook: (1) RecipeViewModel sorts the pantry so that items close to expiry come first. (2) RecipeAiService asks for 3 Turkish recipes as JSON, as shown below. (3) The ViewModel validates the response; if the request or parsing fails, it retries once and then shows a retry message. (4) After Cooked, PantryRepository.markCooked updates the pantry and the list together.

```
{
  "recipes": [{
    "title": "Menemen",
    "minutes": 15,
    "servings": 2,
    "usedIngredients": ["domates", "yumurta", "biber", "soğan"],
    "usesExpiring": ["domates"],
    "missingIngredients": [],
    "steps": ["Biberi ve soğanı kavur.", "Domatesi ekle.", "Yumurtaları kır, karıştır."]
  }]
}
```